

Exercice : Les trois fonctions de convention

Consigne

Écrire les trois fonctions qui fixent la convention une fois pour toutes : `Avant()`, `Haut()` et `Droite()`, qui rendent chacune un vecteur unitaire dans la convention du module. Le programme lit trois réels, les traite comme un point, et affiche son produit scalaire avec chacune des trois directions. Trois lignes, quatre décimales.

Rappel de la convention du module

La convention du module est celle d'OpenXR : main droite, l'avant regarde les z négatifs, $+y$ vers le haut, $+x$ vers la droite.

Les trois fonctions

```
#include <array>

// Convention du module (= OpenXR) :
// main droite, avant = z négatifs, +y = haut, +x = droite.

std::array<float, 3> Avant() { return { 0.0f, 0.0f, -1.0f }; }
std::array<float, 3> Haut() { return { 0.0f, 1.0f, 0.0f }; }
std::array<float, 3> Droite() { return { 1.0f, 0.0f, 0.0f }; }
```

Programme complet

```
#include <array>
#include <cstdio>

std::array<float, 3> Avant() { return { 0.0f, 0.0f, -1.0f }; }
std::array<float, 3> Haut() { return { 0.0f, 1.0f, 0.0f }; }
std::array<float, 3> Droite() { return { 1.0f, 0.0f, 0.0f }; }

float ProduitScalaire(const std::array<float, 3>& a,
                     const std::array<float, 3>& b)
{
    return a[0] * b[0] + a[1] * b[1] + a[2] * b[2];
}

int main()
{
    float x, y, z;
    std::scanf("%f %f %f", &x, &y, &z);

    std::array<float, 3> point = { x, y, z };

    std::printf("%.4f\n", ProduitScalaire(point, Avant()));
    std::printf("%.4f\n", ProduitScalaire(point, Haut()));
    std::printf("%.4f\n", ProduitScalaire(point, Droite()));

    return 0;
}
```

Exemple d'exécution

Pour l'entrée 1.0 2.0 3.0 :

```
-3.0000  
2.0000  
1.0000
```

Explication

- **Avant()** rend $(0, 0, -1)$: l'avant regarde les z négatifs.
- **Haut()** rend $(0, 1, 0)$: $+y$ vers le haut.
- **Droite()** rend $(1, 0, 0)$: $+x$ vers la droite.
- Le produit scalaire d'un point avec **Avant()** donne la composante du point selon l'axe avant. Pour $(1, 2, 3)$, cela donne -3 : le point est derrière l'origine (car l'avant est en z négatif).
- Le produit scalaire avec **Haut()** donne 2 : le point est à 2 au-dessus du sol.
- Le produit scalaire avec **Droite()** donne 1 : le point est à 1 à droite de l'origine.

À retenir

Une convention ne se retient pas, elle s'écrit dans une fonction. Le jour où vous tapez `position.z -= vitesse` en vous demandant si le signe est bon, arrêtez-vous et écrivez **Avant()**. Vous poserez la question une fois, et la réponse sera au même endroit pour tout le programme.